

IOT-BASED SMART AGRICULTURE OVER 5G

by

DEEPAK A	23BEC1220
NITHIN V	23BEC1346
PRANAV S	23BEC1350

A project report submitted to

Dr. VYDEKI D

SCHOOL OF ELECTRONICS ENGINEERING

in partial fulfilment of the requirements for the course of

BECE317L – Wireless and Mobile Communications

in

B.Tech. Electronics and Communication Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Vellore Institute of Technology (VIT) CHENNAI

Vandalur – Kelambakkam Road

Chennai – 600127

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report entitled “**IOT-BASED SMART AGRICULTURE OVER 5G**” is a bonafide work of **DEEPAK A (23BEC1220)**, **NITHIN V (23BEC1346)** and **PRANAV S (23BEC1350)** who carried out the Project work under my supervision and guidance.

Dr. VYDEKI D

Professor

School of Electronics Engineering (SENSE),

VIT University, Chennai

Chennai – 600 127.

ABSTRACT

This project presents the development of an IoT-based smart agriculture monitoring system that leverages cellular connectivity for real-time data transmission and remote access. The system integrates multiple environmental sensors, including a rain sensor, LDR, and ultrasonic sensor, with a Raspberry Pi acting as the central processing unit. These sensors continuously collect data related to rainfall, light intensity, and water levels, enabling efficient monitoring of agricultural conditions.

A key feature of the system is the use of a 5G/LTE HAT, which provides reliable wireless communication by connecting the Raspberry Pi to a cellular network. This ensures uninterrupted data transmission even in areas where traditional Wi-Fi connectivity is limited or unavailable. The collected sensor data is transmitted to the ThingSpeak cloud platform using HTTP protocols, where it is stored, processed, and visualized through a user-friendly dashboard for real-time monitoring.

The system is designed to support intelligent decision-making in agriculture, such as preventing overwatering, optimizing lighting conditions, and monitoring water resources effectively. By combining IoT technology with cellular communication, the project demonstrates a scalable and practical solution for modern precision farming.

ACKNOWLEDGEMENT

We wish to express our sincere thanks and deep sense of gratitude to our project guide, **Dr. VYDEKI D**, Professor, School of Electronics Engineering, for his consistent encouragement and valuable guidance offered to us in a pleasant manner throughout the course of the project work.

We are extremely grateful to **Dr. A. Ravi Sankar**, Dean of the School of Electronics Engineering, VIT Chennai, for extending the facilities of the School towards our project and for her unstinting support.

We express our thanks to our Head of the Department **Dr. M. Jagannath** for his support throughout the course of this project.

We also take this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

DEEPAK A

A. Deepak

NITHIN V

Nithin

PRANAV S

S. Pranav

TABLE OF CONTENTS

SERIAL NO.	NAME	PAGE NO.
	ABSTRACT	3
	ACKNOWLEDGEMENT	4
1	INTRODUCTION	6
	1.1 OBJECTIVES AND GOALS	6
	1.2 BENEFITS	6
	1.3 FEATURES	6
2	DESIGN	7-13
	2.1 BLOCK DIAGRAM	7-8
	2.2 HARDWARE ANALYSIS	8
	2.3 PROCESSING UNIT	8-9
	2.4 SENSOR INPUT STAGE	9-12
	2.5 5G INTEGRATION	12
	2.6 5G HAT WORKING	13
	2.7 SIM CARD & ANTENNA	13
3	SOFTWARE IMPLEMENTATION	14-16
5	NETWORK ANALYSIS	17-21
4	CONCLUSION AND FUTURE WORK	22
	4.1 CONCLUSION	22
	4.2 FUTURE WORK	22
5	REFERENCES	24

1. INTRODUCTION

1.1 OBJECTIVES AND GOALS

- To develop a reliable system that monitors environmental parameters such as light intensity, rainfall, and water levels using appropriate sensors.
- To integrate sensors with a Raspberry Pi for efficient data acquisition and processing.
- To establish seamless wireless communication using a 5G/LTE HAT for transmitting data over cellular networks.
- To implement cloud-based data storage and visualization using the ThingSpeak platform for real-time monitoring.

1.2 BENEFITS

The proposed system offers significant advantages in modern agricultural practices by enabling real-time monitoring of key environmental parameters such as rainfall, light intensity, and water levels. This facilitates efficient water management by preventing overwatering and reducing resource wastage. With cloud integration, the system allows remote access to data, eliminating the need for constant on-site supervision. The use of 5G/LTE connectivity ensures reliable communication even in remote areas where traditional Wi-Fi networks may not be available. Additionally, the system contributes to improved crop productivity by helping optimize growing conditions and supports future automation of irrigation and lighting systems. Its scalable design allows for easy expansion with additional sensors, while also being cost-effective by minimizing manual effort and promoting data-driven decision-making in agriculture.

1.3 FEATURES

- Integration of multiple sensors such as LDR, rain sensor, and ultrasonic sensor to monitor environmental conditions
- Raspberry Pi used as a central processing unit for data acquisition and control
- 5G/LTE HAT module for reliable wireless communication over cellular networks
- Real-time data transmission to cloud using HTTP protocol
- Cloud platform (ThingSpeak) for data storage, analysis, and visualization
- Ultrasonic sensing for accurate water level or distance measurement

- Automated monitoring of rainfall to support smart irrigation decisions
- Light intensity detection to analyze day/night cycles and optimize plant growth
- Efficient and modular Python-based programming for system control and communication
- Scalable architecture allowing easy addition of more sensors and features in future

2. SMART AGRICULTURE OVER 5G SYSTEM DESIGN

2.1 BLOCK DIAGRAM

The block diagram illustrates the architecture of the IoT-based smart monitoring system. Fig. 1 shows the integration of physical sensors such as LDR, rain sensor, and ultrasonic sensor with the Raspberry Pi through GPIO pins. Physical sensors, including the LDR, rain sensor, and ultrasonic sensor, collect environmental data and feed it to the Raspberry Pi through GPIO pins. The Raspberry Pi processes this data using a Python script, applies control logic, and generates HTTP requests for data transmission. A Waveshare 5G HAT with a 5G SIM card enables wireless communication over the 5G network, supported by external antennas. The transmitted data is then analyzed using Wireshark, where packet capture, protocol hierarchy, and network performance metrics are monitored for evaluation.

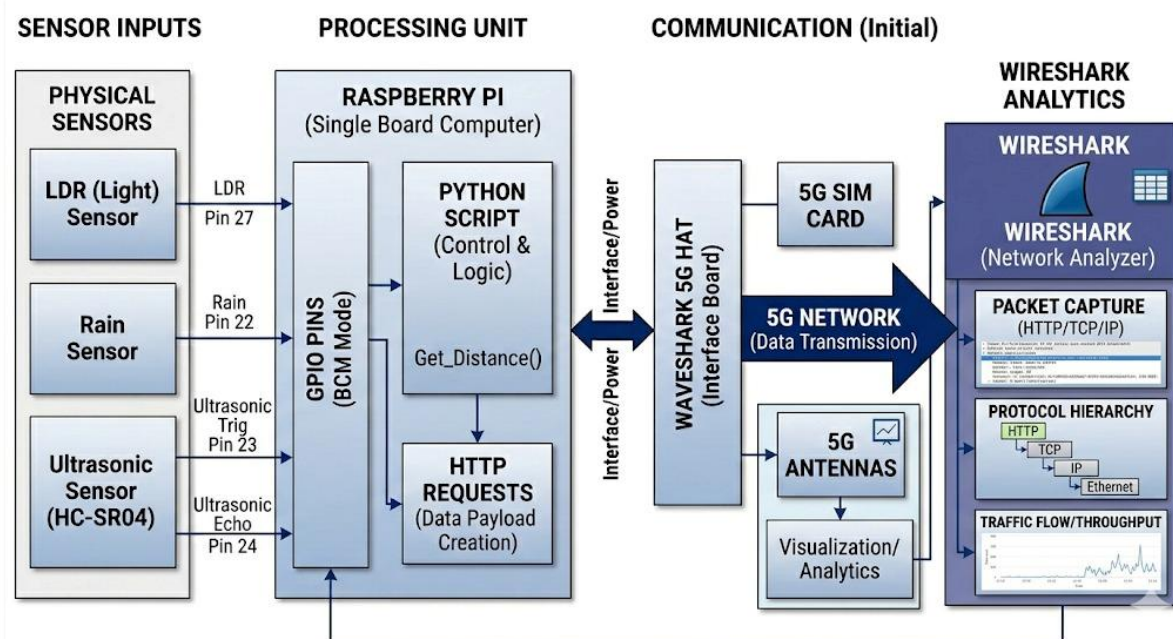


Fig. 1. Block diagram of the IoT-based smart monitoring system integrating sensor inputs (LDR, rain, and ultrasonic), Raspberry Pi processing with Python, 5G communication via Waveshare 5G HAT, and network performance analysis using Wireshark.

2.2 HARDWARE ANALYSIS AND INTERFACING

This project utilizes a multi-sensor array interfaced with a central processing unit (CPU), which then transmits data over a high-speed 5G cellular network to a cloud analytics platform. The hardware architecture can be broken down into three primary stages: the Sensor Input Stage, the Processing and Communication Stage, and the Network/SIM Interface Stage. This section details the working principles of each key hardware component and its implementation in the system. The overall experimental setup and hardware arrangement are illustrated in Fig. 2, which shows the practical implementation of the system components.

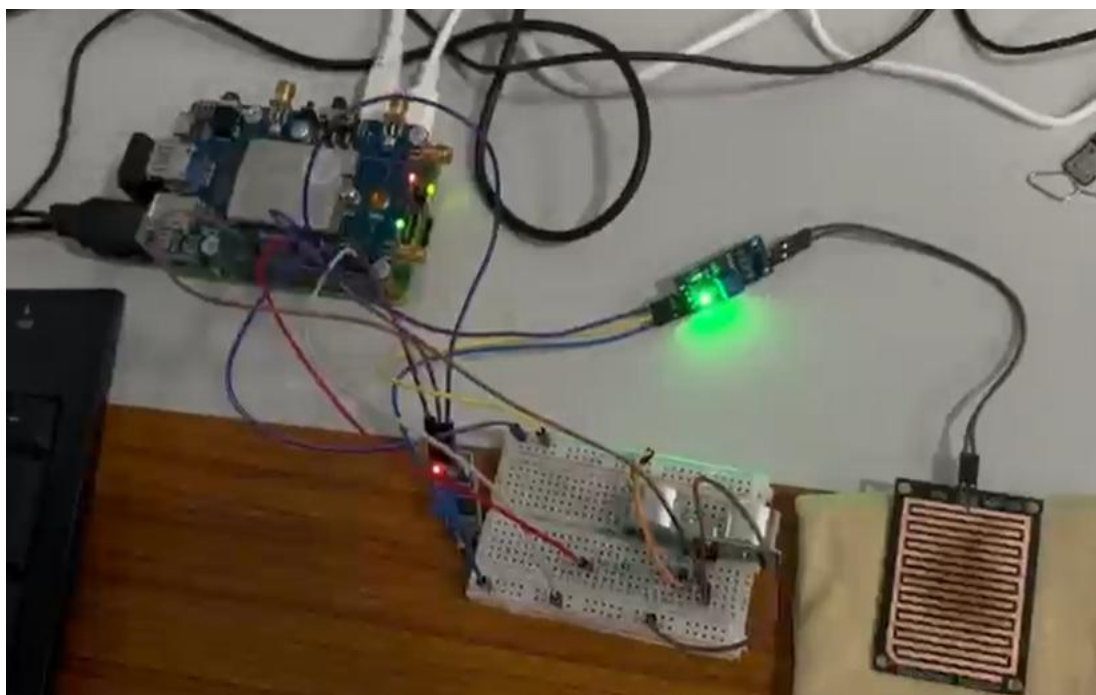


Fig. 2. Experimental setup of the IoT-based smart monitoring system

2.3. PROCESSING AND INTERFACE UNIT

The core of the hardware system is the **Raspberry Pi (SBC - Single Board Computer)**. It is responsible for all control logic, reading raw sensor data, processing calculations (such as distance), and formatting data for cellular transmission. The central processing unit used in the system is shown in Fig. 3, highlighting the Raspberry Pi 4 Model B used for data processing and control.

The Raspberry Pi's **GPIO (General Purpose Input/Output)** pins, configured in BCM (Broadcom) mode, serve as the physical interface for the sensors.

- **Role:** CPU and main system coordinator. It executes the Python control script.
- **Interfacing:** It manages specific pins (Pin 27, Pin 22, Pin 23, Pin 24) to interact with the input devices and a high-level UART/USB interface to communicate with the 5G HAT.

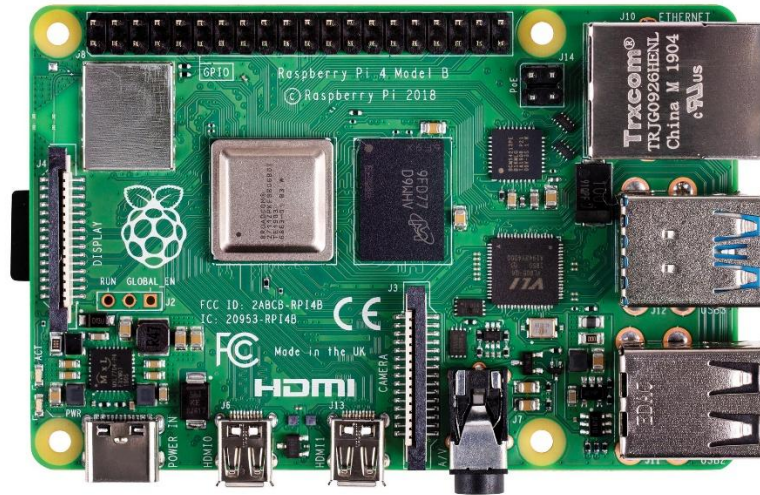


Fig. 3. Raspberry Pi 4 Model B single-board computer

2.4. SENSOR INPUT STAGE

The system utilizes three distinct sensors to monitor environmental and physical conditions.

2.4.1 LDR (LIGHT DEPENDENT RESISTOR) SENSOR

The Light Dependent Resistor (LDR), often integrated with an analog comparator circuit on a sensor module, is used to detect the presence or absence of light. The LDR sensor module used in this system is depicted in Fig. 4, which shows the light detection component interfaced with the Raspberry Pi.

- **Working Principle:** An LDR is a photoresistor whose resistance decreases as the intensity of incident light increases. In darkness, the LDR has very high resistance (millions of ohms); in bright light, it has very low resistance (hundreds of ohms). A module typically converts this analog resistance change into a digital signal using a threshold potentiometer.
- **Implementation:** The LDR module outputs a digital high (no light) or low (light detected) signal. This signal is read directly by the Raspberry Pi's

GPIO Pin 27. The code uses `GPIO.input(ldr_pin)` to read this boolean state.

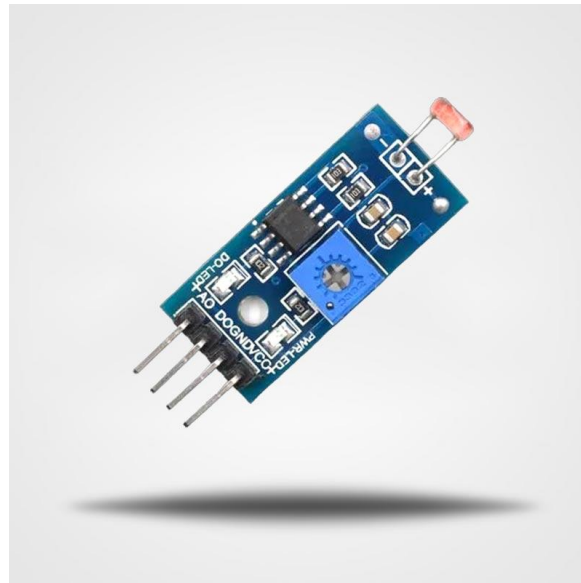


Fig. 4. LDR (Light Dependent Resistor) sensor module used for detecting light intensity in the system.

2.4.2 RAIN/MOISTURE SENSOR

The rain sensor is used to detect precipitation. Like the LDR, it is usually deployed as a sensor probe paired with an electronic module. The rain detection module used for sensing precipitation is shown in Fig. 5, illustrating the probe and control circuitry.

- **Working Principle:** The sensor probe consists of nickel-coated traces designed like a variable resistor. When the probe is dry, its conductivity is low (high resistance). When a raindrop falls on the tracks, it bridges the gap, increasing the conductivity (decreasing resistance). The associated module uses a comparator to output a digital threshold signal.
- **Implementation:** The module provides a digital output (high for dry, low for wet) that is monitored by the Raspberry Pi's GPIO Pin 22.



Fig. 5. Rain sensor module used for detecting the presence of rainfall.

2.4.3 ULTRASONIC DISTANCE SENSOR (HC-SR04)

The HC-SR04 ultrasonic sensor provides non-contact distance measurement from 2 cm to 400 cm. The ultrasonic sensor used for distance measurement is presented in Fig. 6, demonstrating the HC-SR04 module used in the system.

- **Working Principle:** This sensor works by emitting an ultrasonic burst and measuring the time it takes for the echo to return. The process is handled by two pins: Trigger and Echo.
 - The Raspberry Pi sends a short (10 μ s) pulse to the **Trig** pin.
 - The sensor then emits an 8-cycle burst of ultrasound at 40 kHz.
 - The **Echo** pin goes high immediately after the burst is sent.
 - When the reflected signal is detected, the Echo pin goes low.
- **Calculations:** The Pi measures the exact duration the Echo pin remains high. Distance is calculated using the speed of sound:

$$\text{Distance} = \frac{\text{Time Duration} \times \text{Speed of Sound (343 m/s)}}{2}$$

- **Implementation:** The Python code coordinates this timing precisely. GPIO Pin 23 is configured as the Trigger (Output), and GPIO Pin 24 is the Echo (Input). The `get_distance()` function handles the pulse generation and timing calculations.



Fig. 6. HC-SR04 ultrasonic sensor used for measuring distance based on echo time of ultrasonic waves.

2.5. COMMUNICATION STAGE (5G INTEGRATION)

The critical component enabling the 5G functionality of this project is the **Waveshare 5G HAT (Hardware Attached on Top)** board, paired with a 5G SIM card and antennas. The communication module used for enabling cellular connectivity is shown in Fig. 7, which illustrates the Waveshare 5G HAT attached to the Raspberry Pi.

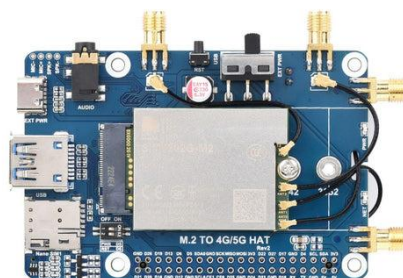


Fig. 7. Waveshare 5G HAT (M.2 to 4G/5G interface board) used for enabling cellular communication between the Raspberry Pi and the server for real-time data transmission.

2.6. WAVESHARE 5G HAT WORKING

The 5G HAT is a specialized communication expansion board designed specifically for the Raspberry Pi.

- **Working Principle:** The HAT contains a high-performance 5G cellular modem (such as the SIM8200EA-M2 or similar, depending on the specific model) mounted on an M.2 interface. It bridges the gap between the Raspberry Pi's local processing and the wide-area cellular network.
- **Interface/Power:** The HAT connects to the Raspberry Pi using the 40-pin GPIO header. While it draws some power from the Pi, high-throughput 5G operation often requires an external power supply connected directly to the HAT to ensure stability. It communicates data with the Pi typically through a high-speed USB 3.0 port (for maximum throughput) or UART (for control).
- **Operation:** When the Python script creates an HTTP request (using `requests.get()`), the Raspberry Pi operating system routes this traffic through the network interface generated by the 5G HAT. The HAT translates these TCP/IP data packets into cellular radio signals.

2.7. SIM CARD AND ANTENNAS

- **5G SIM Card:** The SIM card provides the necessary credentials (IMSI, keys) to authenticate the device on the commercial 5G network, allowing the HAT to register and establish a high-speed data connection.
- **5G Antennas:** Multiple external dipoles or patch antennas are connected to the HAT. These are crucial for MiMo (Multiple Input, Multiple Output) operation, enabling the HAT to send and receive radio RF signals effectively over 5G NR (New Radio) frequencies.

By integrating these components, the hardware system successfully collects analog environmental data, converts it to digital intelligence via the processing unit, and leverages the high bandwidth and low latency of 5G via the Waveshare HAT for real-time cloud uploading.

3. SOFTWARE IMPLEMENTATION

The software used for raspberry pi is python

Program code

```
import RPi.GPIO as GPIO
import time
import requests

# GPIO pins
ldr_pin = 27
rain_pin = 22
trig = 23
echo = 24

API_KEY = "YOUR_API_KEY"

GPIO.setmode(GPIO.BCM)

GPIO.setup(ldr_pin, GPIO.IN)
GPIO.setup(rain_pin, GPIO.IN)

GPIO.setup(trig, GPIO.OUT)
GPIO.setup(echo, GPIO.IN)

print("System started")

def get_distance():
    GPIO.output(trig, False)
    time.sleep(0.5)

    GPIO.output(trig, True)
    time.sleep(0.00001)
    GPIO.output(trig, False)

    while GPIO.input(echo) == 0:
        start = time.time()

    while GPIO.input(echo) == 1:
        end = time.time()

    duration = end - start
```

```

distance = duration * 17150
return round(distance, 2)

while True:
    ldr = GPIO.input(ldr_pin)
    rain = GPIO.input(rain_pin)
    distance = get_distance()

    print("Light:", ldr)
    print("Rain:", rain)
    print("Distance:", distance, "cm")

    url = "https://api.thingspeak.com/update"

    payload = {
        "api_key": API_KEY,
        "field1": ldr,
        "field2": rain,
        "field3": distance
    }

    try:
        requests.get(url, params=payload)
        print("Data sent to ThingSpeak")
    except:
        print("Error sending data")

    time.sleep(15)

```

Code explanation

The implemented Python program is responsible for acquiring sensor data, processing it, and transmitting it to a cloud platform. The code begins by importing the required libraries, namely RPi.GPIO for interfacing with the Raspberry Pi's GPIO pins, time for handling delays and timing operations, and requests for enabling HTTP communication with the cloud server. These libraries collectively support hardware interaction, precise timing, and network communication, which are essential components of the system.

The GPIO pins are then defined for each sensor. The LDR sensor is connected to pin 27, the rain sensor to pin 22, and the ultrasonic sensor uses pins 23 and 24 for trigger and echo respectively. An API key is also defined, which is used to authenticate data transmission to the ThingSpeak platform. The GPIO mode is set

to BCM (Broadcom mode), allowing the use of GPIO pin numbers instead of physical pin numbers, ensuring consistency with the Raspberry Pi pin mapping.

Following this, the GPIO pins are configured based on their functionality. The LDR and rain sensor pins are set as input pins, as they provide digital signals to the Raspberry Pi. For the ultrasonic sensor, the trigger pin is configured as an output to send ultrasonic pulses, while the echo pin is set as an input to receive the reflected signal. Once the setup is complete, a confirmation message is printed indicating that the system has started successfully.

A dedicated function named `get_distance()` is implemented to calculate the distance using the ultrasonic sensor. The function begins by ensuring the trigger pin is low and allowing the sensor to stabilize. A short pulse of 10 microseconds is then sent through the trigger pin, which causes the ultrasonic sensor to emit a sound wave. The program then continuously monitors the echo pin to detect when the signal is sent and received. The time difference between sending and receiving the signal is calculated, which represents the duration taken by the ultrasonic wave to travel to the object and back. This duration is then converted into distance using the speed of sound, and the result is returned in centimeters.

The main execution loop runs indefinitely using a `while True` construct. Within this loop, the system continuously reads the status of the LDR and rain sensors using digital input, and also calls the `get_distance()` function to obtain the distance measurement. These values are printed to the console for real-time monitoring and debugging purposes.

For data transmission, a URL corresponding to the ThingSpeak API is defined. A payload dictionary is created, which includes the API key and the sensor readings mapped to specific fields. The program then attempts to send this data to the server using an HTTP GET request. A try-except block is used to handle potential communication errors, ensuring that the program does not crash if the data transmission fails. Upon successful transmission, a confirmation message is displayed; otherwise, an error message is printed.

Finally, a delay of 15 seconds is introduced at the end of each loop iteration. This delay is necessary to comply with ThingSpeak's rate limits and to ensure stable and periodic data transmission. Overall, the code effectively integrates sensor interfacing, data processing, and cloud communication to realize a functional IoT-based monitoring system.

4. NETWORK ANALYSIS

Wireshark is a widely used open-source network protocol analyzer that enables real-time packet capturing and detailed inspection of network activity across various layers of the TCP/IP model. It provides deep visibility into protocols such as DNS, TCP, UDP, and HTTP/HTTPS, allowing users to examine packet structure, timing, and communication patterns. Wireshark is utilized to capture and analyze network traffic generated by a system communicating with external servers. The tool facilitates the identification of key networking processes such as domain name resolution, connection establishment, data transfer, and session termination. By interpreting captured packets and protocol behavior, meaningful insights into network performance metrics such as latency, throughput, and reliability can be derived.

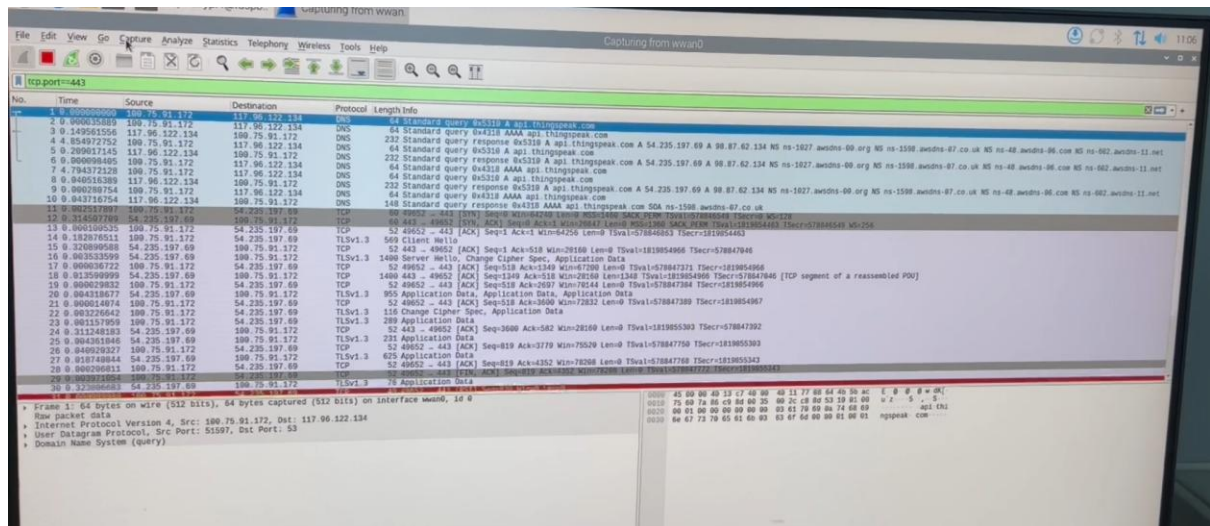


Fig. 8. Wireshark capture showing DNS resolution, TCP three-way handshake, and secure TLS communication between the IoT device and ThingSpeak server.

Fig. 8 shows the Wireshark capture illustrating DNS resolution, TCP handshake, and TLS communication. The captured Wireshark trace illustrates the communication sequence between the embedded board and the ThingSpeak server over TCP/IP. Initially, multiple DNS query and response packets can be observed, where the client resolves the domain name *api.thingspeak.com* into its corresponding IP address (54.235.197.69), indicating proper domain resolution prior to data transmission. Following this, a standard TCP three-way handshake is established, evidenced by the SYN, SYN-ACK, and ACK packets between the source (client device) and destination (ThingSpeak server on port 443). Once the connection is established, TLSv1.3 packets are exchanged, confirming that the communication is secured using HTTPS. The presence of “Application Data” packets indicates that sensor data from the board is being transmitted to the server in encrypted form. Additionally, acknowledgment (ACK)

packets and sequence numbers reflect reliable data delivery and flow control. The session concludes with FIN/ACK packets, demonstrating a termination of the TCP connection.

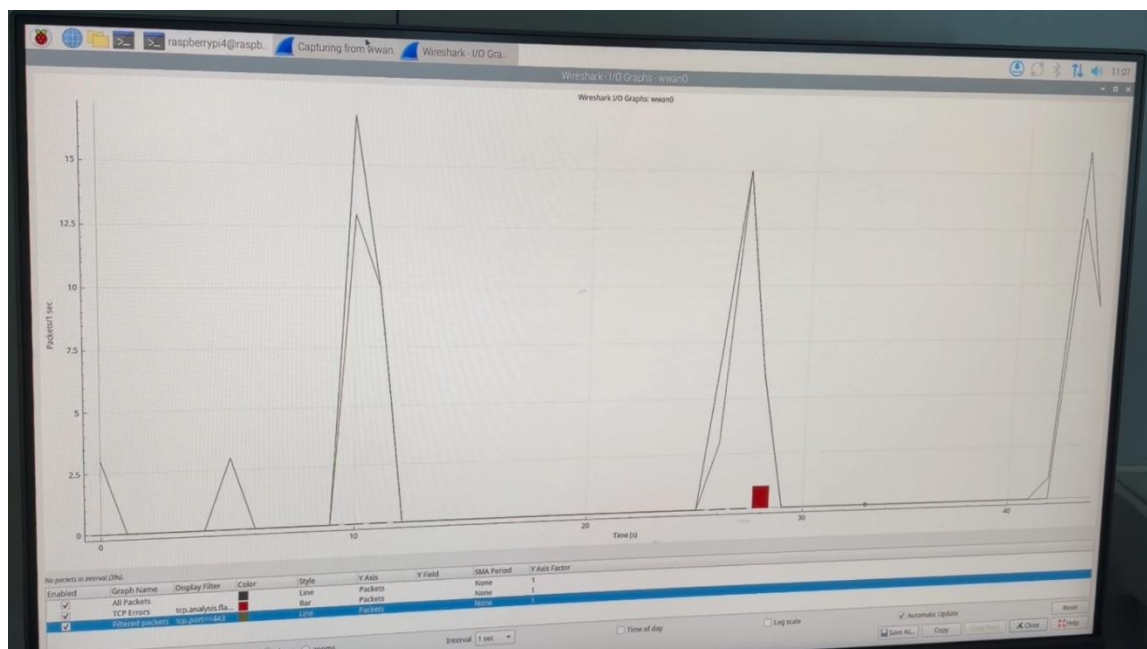


Fig. 9. Wireshark I/O graph illustrating packet transmission rate over time

Fig. 9 illustrates the I/O graph representing packet transmission rate over time, focusing on TCP traffic filtered on port 443, which corresponds to secure HTTPS communication. The graph shows distinct spikes at regular intervals, indicating periodic bursts of data transmission from the device to the server. These peaks represent instances where sensor data is being uploaded, followed by intervals of minimal or no traffic, suggesting idle periods between transmissions.

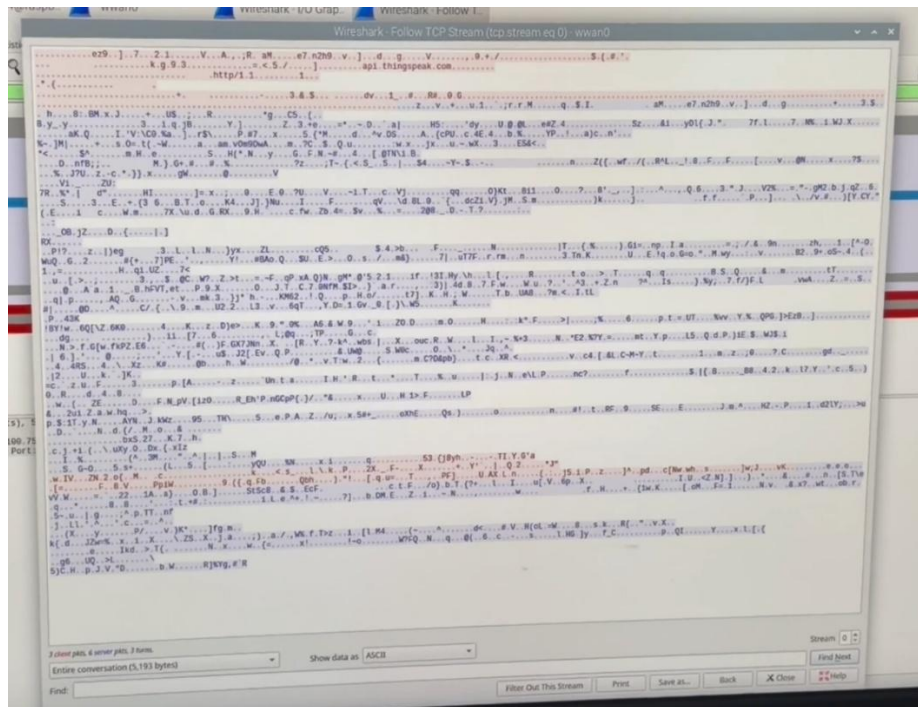


Fig. 10. “Follow TCP Stream” view showing encrypted data exchange between the IoT device and ThingSpeak server.

The displayed TCP stream in Fig 10. represents the actual data transmitted between the client device and the ThingSpeak server during the session. The content appears largely unreadable and filled with random characters, indicating that the payload is encrypted using TLS (Transport Layer Security). This confirms that the data, including sensor readings and HTTP requests, is securely encapsulated before transmission, preventing unauthorized access or interception. The presence of both client-side and server-side data streams (highlighted in different colors) demonstrates bidirectional communication within a single TCP session.

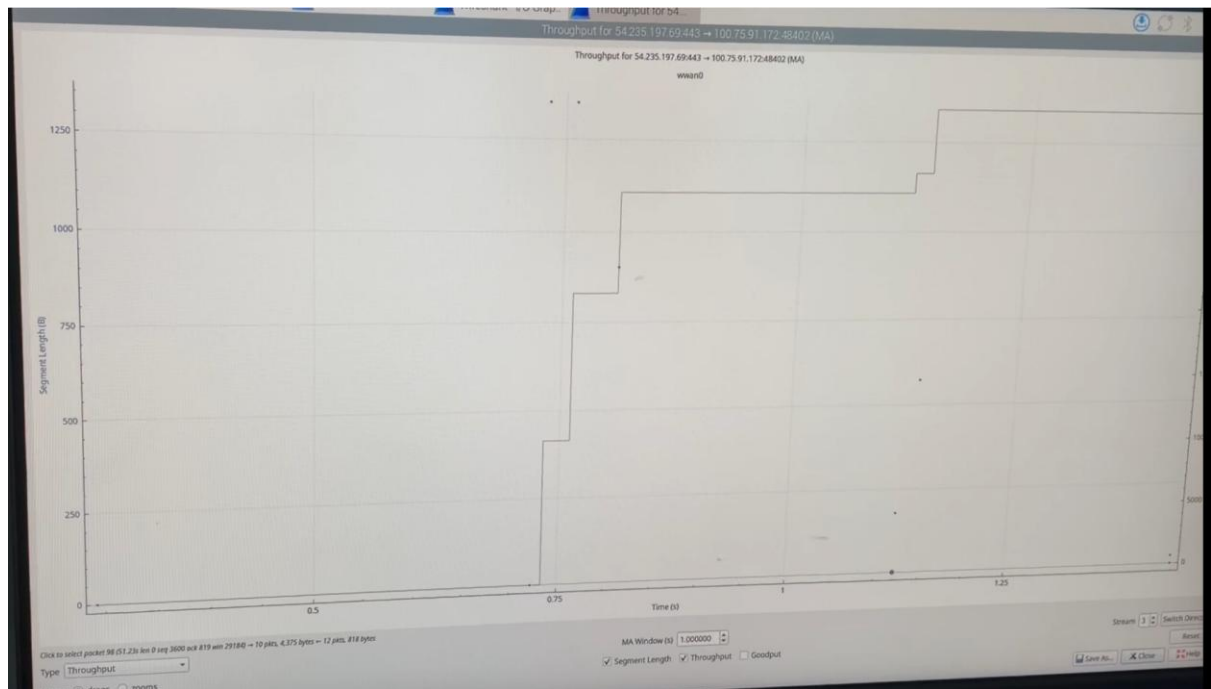


Fig. 11. Wireshark throughput graph showing variation of data transmission (bytes) over time for the TCP connection between the device and ThingSpeak server.

The throughput graph Fig. 11. illustrates the rate at which data is transmitted from the client device to the server over time, measured in terms of segment length (bytes). The step-like increase observed in the graph indicates bursts of data transfer, where packets are sent in chunks rather than continuously. Initially, the throughput remains low, followed by a sharp rise, suggesting the start of significant data transmission after connection establishment. The gradual increments further indicate controlled data flow, governed by TCP congestion control and acknowledgment mechanisms.

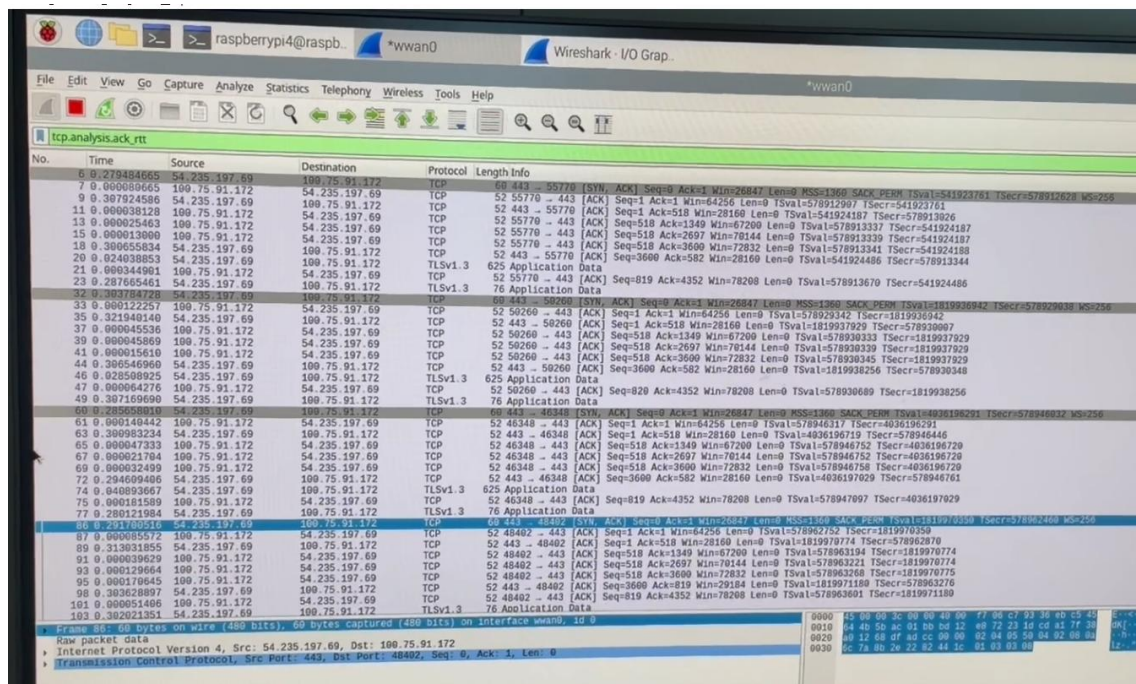


Fig. 12. Wireshark packet list filtered for TCP ACK RTT showing acknowledgment timing between client and server.

Latency Calculation:

Fig. 12 shows the TCP ACK RTT analysis used for latency calculation. The network latency is estimated using the time difference between a transmitted TCP packet and its corresponding acknowledgment (ACK). From the Wireshark capture, a packet is observed at approximately 0.279 s, and the corresponding ACK is received at approximately 0.300 s. The round-trip time (RTT), which represents the network latency, is calculated as:

$$\text{Latency (RTT)} = \text{Time of ACK} - \text{Time of packet sent}$$

$$\text{Latency (RTT)} = 0.300 - 0.279 = 0.021 \text{ seconds}$$

The calculated network latency is approximately 21 ms. This low latency value indicates a stable and efficient network connection, making it suitable for reliable real-time data transmission in IoT applications.

5. CONCLUSION AND FUTURE WORK

4.1 CONCLUSION

- The IoT-based smart agriculture monitoring system was successfully designed and implemented.
- The system integrates multiple sensors with a Raspberry Pi for efficient data acquisition and processing.
- Reliable wireless communication was established using a 5G/LTE HAT for data transmission over cellular networks.
- Real-time sensor data was successfully transmitted to the ThingSpeak cloud platform and visualized through a dashboard.
- The system demonstrates effective monitoring of environmental conditions such as rainfall, light intensity, and water levels.
- The overall performance of the system is stable, scalable, and suitable for smart farming applications.

The network analysis conducted using Wireshark provides insights into the communication behavior between the IoT device and the ThingSpeak server. The captured packets clearly demonstrated the complete workflow of network communication, beginning with DNS resolution, followed by TCP connection establishment through the three-way handshake, and secure data transmission using TLS encryption. The presence of encrypted application data confirmed that the system ensures confidentiality and integrity during data exchange.

Further analysis using I/O graphs and throughput measurements revealed that data transmission occurs in periodic bursts, indicating a structured and efficient data upload mechanism. The absence of significant packet loss, retransmissions, or TCP errors highlights the reliability and stability of the network connection. Additionally, the calculated latency of approximately 21 ms reflects a low-delay communication link, which is suitable for real-time IoT applications.

4.2 FUTURE WORK

- Implementation of automated irrigation control based on sensor data.
- Integration of additional sensors such as temperature, humidity, and soil moisture for enhanced monitoring.
- Development of a mobile application for user-friendly remote access and control.
- Incorporation of machine learning techniques for predictive analysis and smart decision-making.

- Use of solar power systems to make the setup more energy-efficient and sustainable.
- Enhancement of system security using advanced protocols for safe data transmission.
- Expansion of the system for large-scale agricultural deployments with multiple nodes.

5. REFERENCES

- [1] Department of Telecommunications (DoT), 5G Lab Manual, Edition 1, 2025.
- [2] Signaltron Systems Pvt. Ltd., 5G Labs Training Manual, Rev. 2.0, Feb. 2025.
- [3] V. Mahore, P. Patidar, P. Soni, H. Nagar, A. Chouriya and A. Paul, "An IoT-Enabled Tractor Data Sensing System for Precision Agriculture," 2023 2nd International Conference on Futuristic Technologies (INCOFT), Belagavi, Karnataka, India, 2023.
- [4] S. Muthusundari, M. Manuteja, N. Yeshwanth, K. K. Reddy, M. I. Habelalmateen and S. Ritwika, "A Smart Farming : Implementation Of Iot Based Automated Agricultural Monitoring System," 2023 International Conference for Technological Engineering and its Applications in Sustainable Development (ICTEASD), Al-Najaf, Iraq, 2023.
- [5] B. Ramesh, M. Divya and G. P. Revathi, "Farm Easy- IoT based Automated Irrigation, Monitoring and Pest Detection using ThingSpeak for Analysis of Ladies Finger Plant," 2020 International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT), Bangalore, India, 2020.